Solution Review: Highs and Lows

This review explains the solution for the 'Highs and Lows' exercise.

WE'LL COVER THE FOLLOWING

- Solution
- Explanation

Solution

```
def count_low_high(num_list):
    high_list = list(filter(lambda n: n > 50 or n % 3 == 0, num_list))
    low_list = list(filter(lambda n: n <= 50 and not n % 3 == 0, num_list))
    return [len(low_list), len(high_list)]


num_list = [20, 9, 51, 81, 50, 42, 77]

print(count_low_high(num_list))
```

Explanation

In this solution, the star of the show is the `filter()` function. We use it to filter all the elements that count as a `high` in one list and all the rest in another list.

To count all the elements of both lists, we can simply use the `len` function.

We didn't have to use lambdas but they definitely make the code simpler, and that's what Python is all about!

What is the Python Standard Library?

In this section, we will explore the Python Standard Library.

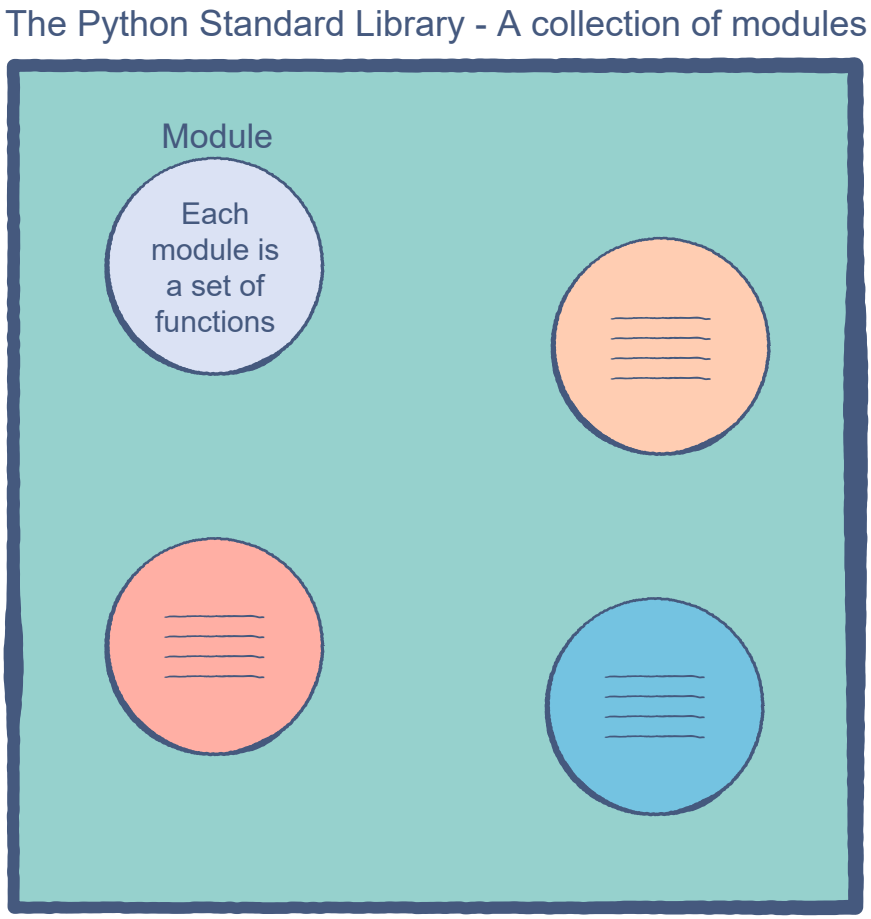
WE'LL COVER THE FOLLOWING

- Definition
- Importing a Module

Definition

> The Python Standard Library, PSL, is a collection of pre-defined **modules**, or sets of **methods** which help us perform certain tasks in Python.

The library contains many useful utilities which spare us a lot of time. There are different sorts of complex mathematical functions, high-level data types, network programming tools, and this is just the tip of the iceberg!



Generally, a module contains functions related to a particular aspect of programming. This makes things easy because we know which part of our program requires which module.

In Python, we can make our own modules, but that's a story for another time. We'll be focusing on what Python offers us out-of-the-box.

Importing a Module

To use the methods of a module, we must import the module into our code. This can be done using the `import` keyword.

Let's import the `datetime` module which contains several methods related to the current date and time. As always, these methods can be accessed using the `.` operator:

```
# Importing modules at the beginning of the program is a good practice

import datetime

date_today = datetime.date.today()  # Current date
print(date_today)
```

```
time_today = datetime.datetime.now()
print(time_today.strftime("%H:%M:%S"))  # Current time
```

In the code above, `datetime.date` and `datetime.datetime` are classes in the `datetime` module. Each class contains its own methods.

If we only want a particular class from a module, we can use the `from` keyword in the following format:

```
from module import class
```

Let's try this out:

```
from datetime import date

# Now we only have the methods of the date class
date_today = date.today()  # Current date
print(date_today)

# These won't work
# time_today = datetime.datetime.now()
# print (time_today.strftime("%H:%M:%S"))# Current time
```

We can also give our own names to the modules that we import by using the `as` keyword. Let's rename `datetime` to `dt` and use it in the program:

```
import datetime as dt

date_today = dt.date.today()  # Current date
print(date_today)

time_today = dt.datetime.now()
print(time_today.strftime("%H:%M:%S"))  # Current time
```

We've learned how to import and use PSL modules in our code. In the next lesson, we'll look at some popular modules.

Popular Modules

This lesson highlights some of the most commonly used Python modules.

WE'LL COVER THE FOLLOWING

- `math`
- `heapq`
- `random`

Here are some useful Python modules for beginners. Keep in mind that this is by no means an exhaustive list. For the full Python library module list, check out their official documentation.

`math`

The `math` module offers a wide range of mathematical functions such as factorial, trigonometric operations, etc.

```
import math

fact_of_5 = math.factorial(5)  # The factorial of 5
print(fact_of_5)

gcd = math.gcd(300, 90)  # Greatest common denominator
print(gcd)

log100 = (math.log(10, 100))  # Logarithm of 10 to the base of 100
print(log100)
```

To get all `math` methods for complex numbers, use the `cmath` module instead.

`heapq`

The `heapq` module allows us to create the **heap** data structure. A heap is a

binary tree which always stores a special value at the top (root). A **minheap**

stores the smallest value at the top and a **maxheap** stores the largest value at the top.

The `pop` method returns the value at the top of the heap.

Python's `heapq` creates a minheap by default.

```
import heapq

heap = []  # Empty heap

# Inserting elements in the heap
heapq.heappush(heap, 10)
heapq.heappush(heap, 70)
heapq.heappush(heap, 5)
heapq.heappush(heap, 35)
heapq.heappush(heap, 50)

# Popping the smallest value
minimum = heapq.heappop(heap)
print(minimum)
```

`random`

The `random` module is used for generating random numbers in Python. There are several methods which allow us to generate different types of random numbers.

The `random()` method generates a random floating-point number between `0` and `1`, whereas `uniform()` returns a floating-point number within a custom range.

```
import random

rand_num = random.random()
print(rand_num)

rand_num_in_range = random.uniform(30, 50)  # A random number between 30 and 50
print(rand_num_in_range)

str_list = ['a', 'b', 'c', 'd', 'e']
random.shuffle(str_list)  # Randomly shuffle a list
print(str_list)
```

The library is filled with many crafty modules like the ones mentioned above. Be sure to check it out.

Apart from the Standard Library, there are thousands of reliable external packages which can be integrated into Python. We'll talk about them in the next lesson.

PyPI

This lesson explains what Python packages are.

WE'LL COVER THE FOLLOWING

- What are Packages?
- Popular Packages

What are Packages?

We have previously studied that Python has a standard library which contains many different modules which serve different purposes. However, we are not limited to these modules.

Developers from all over the world have contributed to increasing the functionality of Python through **packages**.

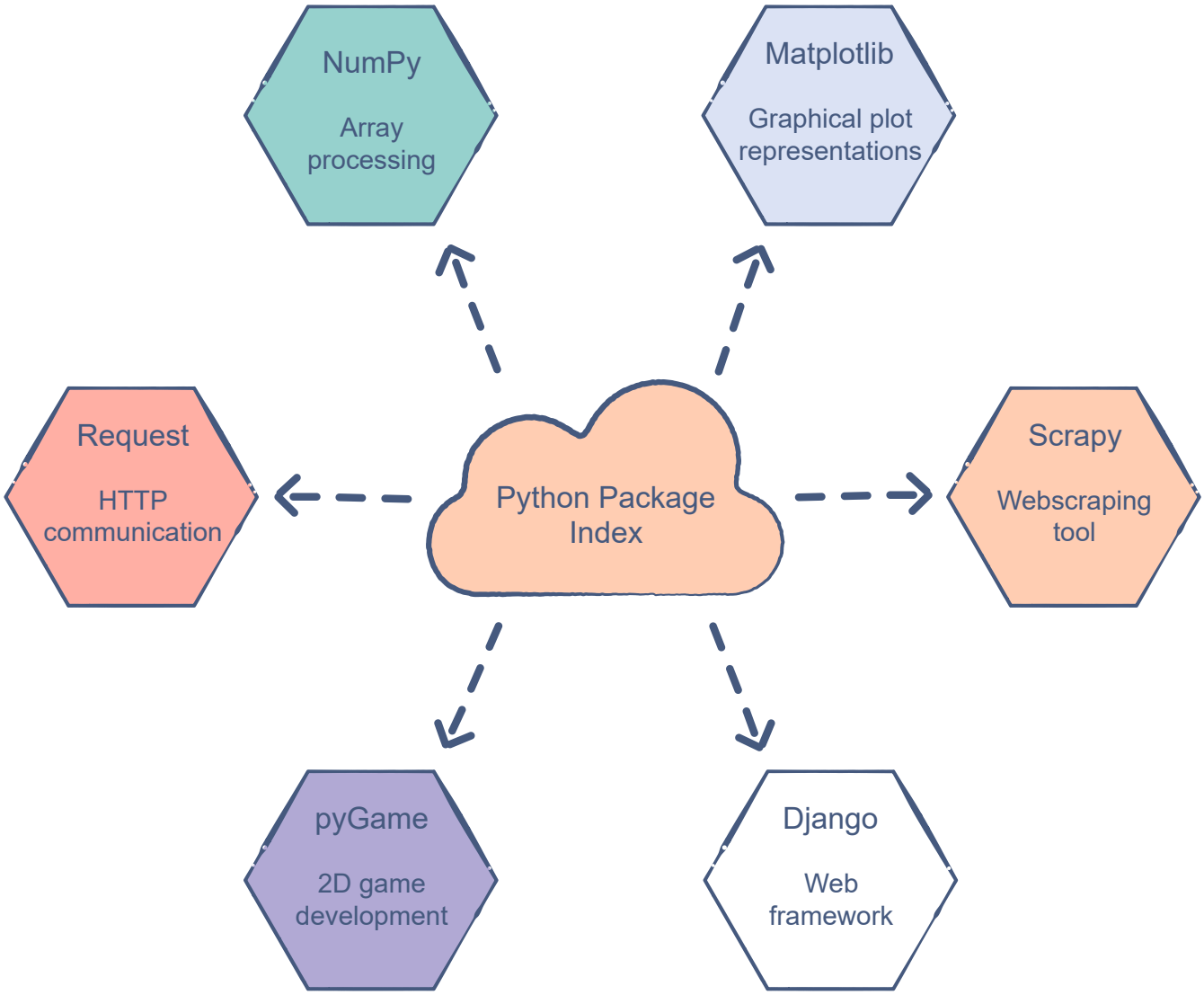
Packages are also collections of modules or smaller packages that assist us in performing certain tasks. The only difference is that these packages have to be installed before they can be imported.

All of these packages can be found at the Python Package Index, PyPI.

There are packages for creating neural networks, text processing, cryptography, statistical modelling and plotting, you name it!

Popular Packages

Here are some of the most popular Python packages.



You can explore the countless other packages on the official website linked above.

In the next lesson, we'll conclude this course and take a look at what lies ahead.

Conclusion

A few final words.

WE'LL COVER THE FOLLOWING

- What Lies Ahead

Here we are, at the end of our long journey. Hopefully, you have a good understanding of how all the basic mechanics of Python work.

What Lies Ahead

This is just the tip of the iceberg in regard to Python programming. You are now equipped with all the building blocks of complex programs. However, learning how to create those programs is still a few steps away.

The next step is to understand object-oriented programming in Python, for which we have a brilliantly effective course!

If you want to explore the applications of Pythonic data structures further, check out the Data Structures in Python course.

Happy coding!